2

Advanced Combinational Logic Circuits with FPGA

2.1 Introduction

This chapter examines complex combinational circuits constructed exclusively with **combinational always blocks** in Verilog.

Table 2.1: Commonly Used Verilog Operators

Verilog Operator	Name
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`~`	Bitwise NOT
`+`	Arithmetic Addition
`-`	Arithmetic Subtraction
`*`	Arithmetic Multiply
`/`	Arithmetic Division
`%`	Modulo
`<<`	Logical Left Shift
`>>`	Logical Right Shift
`{}`	Concatenation

2.2 Preliminary Work

- Review combinational always block (`always@(*)`) design in Verilog, emphasizing `if-else` structures and `switch-case` statements.
- Examine the Basys 3 board datasheet to understand device-specific details. Study pin constraints for switches, LEDs, and 7-segment displays.
- Study different Verilog operators and common anode seven segments.

2.3 8:1 Multiplexer (20 Points)

Construct an 8:1 multiplexer using an `always @(*)` block with `if-else` (or nested `if` statements) to route one of eight inputs (`in[7:0]`) to a single output `y`. The 3-bit select lines `sel[2:0]` determine which input line is forwarded. Do not use `switch-case` statements, systematically check values of `sel` from `0` to `7` instead.

2.4 BCD to 7-Segment Decoder (40 Points)

Design a 4-bit BCD-to-7-segment decoder that drives the display segments `a-g`. Each segment is derived from an `always @(*)` block containing `case` statements for the digits 0–9. If the BCD value is invalid (greater than 9), drive all segments to an invalid pattern (all on or all off, depending on the active polarity). Document the mapping from each digit to the `a`–`g` outputs.

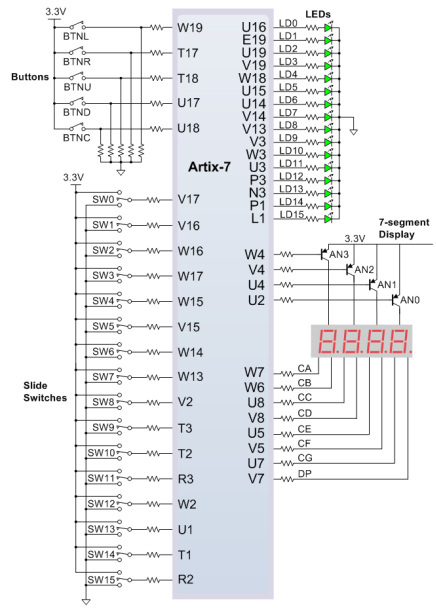

Fig. 2.1: Basys 3 I/O overview.

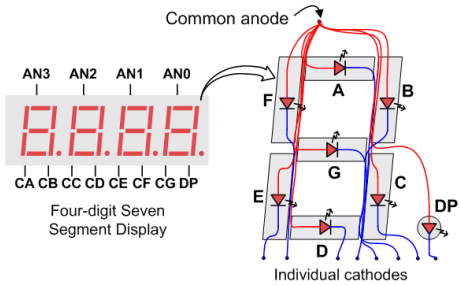

Fig. 2.2: Seven-segment display connections on Basys 3.

Review the constraints file for the Basys 3 board to identify which pins connect to each segment and how to assign 4 bits to the BCD input. Map these signals to your HDL module to test the decoder on real hardware.

2.5 Various Combinational Circuits (40 Points)

Different student groups will work on one or more of the following circuits according to their session schedule:

Monday 2nd, 13:30–15:30 → 3:8 Decoder (Sec. 2.5.2)
Tuesday 3rd, 13:30–15:30 → Priority Encoder (Sec. 2.5.1)
Thursday 5th, 13:30–15:30 → 4-Bit Comparator (Sec. 2.5.3)
Friday 6th, 14.00–16.00 → 4-Bit Adder-Subtractor (Sec. 2.5.4)

2.5.1 Priority Encoder (8:3)

Create a priority encoder with inputs `in[7:0]`. The output `out[2:0]` reflects the highest active input line. Use nested `if-else` statements, starting with `in[7]` down to `in[0]`. Default to `000` if no lines are active. Table 2.2 describes the behavior:

Table 2.2: 8:3 Priority Encoder Behavior

X7	X6	X5	X4	X3	X2	X1	X0	out[2:0]
1	-	-	-	-	-	-	-	111
0	1	-	-	-	-	-	-	110
0	0	1	-	-	-	-	-	101
0	0	0	1	-	-	-	-	100
0	0	0	0	1	-	-	-	011
0	0	0	0	0	1	-	-	010
0	0	0	0	0	0	1	-	001
0	0	0	0	0	0	0	1	000

2.5.2 3:8 Decoder

Build a 3:8 decoder with input `in[2:0]` and outputs `out[7:0]`. Only one line in `out` is high for a given input. Use a single `always @(*)` block with `if-else` conditions to match each possible input value.

2.5.3 4-Bit Comparator

Create a comparator for `a[3:0]` and `b[3:0]`. The outputs are `gt`, `eq`, and `lt`. Use `if-else` to set these signals according to whether `a` is greater than, equal to, or less than `b`.

2.5.4 4-Bit Adder-Subtractor (Based on Cin)

Implement a module that computes `a[3:0]`+`b[3:0]` if `cin` $= 0$, or `a[3:0]`+ $\sim$ `b[3:0]` $+ 1$ if `cin` $= 1$. Produce a 4-bit sum `sum[3:0]` and a single-bit `cout`. No `case` statement is allowed. A single `if-else` on `cin` can handle both operations.

2.6 FPGA Implementation and Testing

- Write all modules in `advanced_circuits.v`. Provide a corresponding testbench for each module to verify correctness.
- Synthesize using the Basys 3 board settings. Map inputs to DIP switches or buttons, and outputs to LEDs or the 7-segment display as needed.
- Check and record the actual hardware outputs to confirm correct functionality.

2.7 Report Requirements

- Provide block diagrams or circuit schematics for each design, clarifying signal flow.
- Include pictures of the programmed FPGA.
- Supply truth tables or text-based logic definitions for all inputs and outputs.
- Include simulation waveforms or logs that confirm correct behavior across valid input ranges.
- Present synthesis reports indicating resource usage, such as LUT counts and carry logic utilization.
- Note any unique challenges or observations encountered during implementation or testing.